

INFORME DE PROYECTO FINAL

Asignatura: Programación Orientada a Objetos (POO)

Proyecto: AlcanclA

Equipo de Desarrollo: Santiago Agostini, Avril Ogas, Lorenzo Poletto

1. Resumen Ejecutivo y Objetivos

El presente documento detalla el análisis, diseño, arquitectura y desarrollo de **AlcanclA**. Este sistema es una plataforma de gestión financiera personal construida en **C++** mediante el framework **Qt 6**, complementada por un backend en la nube desarrollado en **Python (FastAPI)** y una base de datos centralizada en **MySQL**.

Objetivo Principal: Solucionar la desorganización financiera personal eliminando la carga manual de comprobantes físicos y previniendo el cobro imprevisto de servicios recurrentes, actuando como un contador virtual proactivo.

Pilares Tecnológicos del Proyecto:

- **Digitalización Automatizada (OCR + IA):** Extracción inteligente de datos desde fotografías y facturas digitales (PDF) utilizando el modelo GPT-4o-mini.
- **Categorización Contextual:** Clasificación semántica de gastos en categorías predefinidas de manera automática.
- **Gestión Predictiva de Vencimientos:** Monitoreo activo de fechas de renovación con alertas visuales dinámicas.
- **Arquitectura Híbrida (Offline-First):** Sincronización asíncrona bidireccional entre un caché local de altísima velocidad (SQLite) y un servidor remoto transaccional (MySQL).

2. Aplicación de Conceptos de Programación Orientada a Objetos (POO)

El sistema ha sido estructurado respetando rigurosamente los pilares de la POO, lo cual garantiza la escalabilidad, mantenibilidad y la ausencia de código duplicado, cumpliendo con los estándares de evaluación de la asignatura.

2.1. Abstracción y Modelado de Datos

Las entidades del dominio financiero se abstraieron en estructuras nativas de C++. Se creó un modelo donde cualquier transacción se define por atributos esenciales (monto, fecha, categoría), permitiendo escalar el sistema a futuros tipos de movimientos.

2.2. Herencia

Se implementó una jerarquía de clases estricta para establecer una relación *"Es-un"* y fomentar la reutilización de código:

- **Clase Base (MovimientoBase):** Define los cimientos del sistema. Contiene los atributos id, monto, fecha, categoria y descripcion.
- **Subclase Ticket:** Especializa a la clase base agregando atributos físicos del comprobante como el nombreLocal, la imagenPath y una lista compuesta por los productos comprados (ItemTicket).
- **Subclase Suscripcion:** Extiende la clase base sumando propiedades de control temporal como fechaVencimiento, diasAviso y el estado lógico activa.

2.3. Polimorfismo

El polimorfismo se aplica activamente extendiendo y sobrescribiendo el comportamiento del framework Qt:

- **Sobrescritura de métodos virtuales:** En los componentes UploadTicketDialog y AddSubscriptionDialog, se sobrescribe el método virtual accept() heredado de QDialog. Esto permite interceptar el evento de cierre, inyectar validaciones de negocio e invocar a la base de datos antes de destruir la ventana.
- **Custom Painting:** Se sobrescribe el evento paintEvent(QPaintEvent*) en la clase gráfica BarChart para renderizar polígonos, gradientes y textos personalizados en los reportes estadísticos, logrando un control visual absoluto.

2.4. Encapsulamiento

El estado interno de la aplicación se encuentra fuertemente protegido, separando la capa de presentación (GUI) de la capa de datos y red:

- **Capa de Persistencia:** La clase adminDB oculta la complejidad del motor SQLite. Mantiene privada la conexión QSqlDatabase y expone únicamente métodos públicos controlados como sincronizarDesdeJson() o generarNotificacionesVencimiento().
- **Patrón de Diseño Singleton (DataManager):** El coordinador central de la aplicación (DataManager) restringe su instanciación haciendo su constructor privado. Esto garantiza un único punto de acceso en memoria (DataManager::instance()) para administrar la cola asíncrona de QNetworkAccessManager, protegiendo la sesión del usuario de posibles colisiones de red.

3. Diseño de la Interfaz Gráfica y Experiencia de Usuario (UI/UX)

La interfaz de Alcancia fue desarrollada poniendo foco en la usabilidad, la navegación intuitiva y la solidez ante errores de usuario.

- **Diseño Visual Profesional:** Se abstraigo todo el apartado gráfico en una clase estática StyleManager. Ésta provee Hojas de Estilo en Cascada (QSS) que dotan al sistema de un "Modo Oscuro" estético, con sombras reactivas al cursor (hover), bordes redondeados y una paleta de colores cohesiva.
- **Validación Rigurosa de Entradas:** El sistema bloquea acciones indebidas proactivamente. En la creación de suscripciones o carga de tickets, se valida que los textos no estén vacíos y los montos sean mayores a cero, emitiendo un QMessageBox informativo para guiar al usuario.
- **Navegación Eficiente:** Se implementó un menú lateral (Sidebar) conectado a un QStackedWidget, permitiendo transiciones fluidas e instantáneas entre los módulos de Dashboard, Tickets, Suscripciones y Reportes.

4. Arquitectura de Infraestructura y Sincronización

El sistema adopta una topología de red moderna que permite a la aplicación de escritorio operar incluso ante cortes de internet.

Mecanismo de Interacción (Offline-First):

1. **App de Escritorio (Qt 6):** Recolecta los datos y permite visualización instantánea leyendo de su caché.
2. **Caché Local (SQLite):** Archivo local tasty_alcancia.db. Resguarda los hash de contraseñas para logueos sin red, y mantiene una copia de trabajo de los gastos e ítems del usuario activo.
3. **API Intermediaria (FastAPI):** Escucha las peticiones en el VPS, encripta datos y rutea los archivos adjuntos.
4. **Base de Verdad (MySQL):** Motor relacional centralizado en el servidor que garantiza la integridad a largo plazo (ACID) y evita pérdida de datos si la PC del usuario se daña.

5. Backend, API e Integración con Inteligencia Artificial

El cerebro de la aplicación reside en un servidor VPS con Ubuntu 24.04 LTS, gestionado íntegramente mediante contenedores Docker para aislar la API de Python y el motor MySQL.

5.1 Procesamiento Inteligente de Comprobantes (OCR + IA)

El flujo de reconocimiento automático elimina la carga manual:

- El cliente C++ empaqueta la foto o el PDF en formato QHttpMultiPart y lo envía vía POST.
- El backend en Python utiliza PyMuPDF para transformar documentos PDF a imágenes en memoria RAM.
- La imagen codificada se envía a la API de **OpenAI (GPT-4o-mini)** con un "System Prompt" estricto, obligando al modelo a responder exclusivamente con una estructura JSON válida que contiene el comercio, fecha, ítems desglosados y la sugerencia semántica de la categoría.

5.2. Borrado Lógico (Soft Delete) y Preservación Histórica

Garantizando la integridad contable, el sistema nunca ejecuta instrucciones DELETE destructivas sobre movimientos financieros. En su lugar:

- C++ marca el registro en SQLite con `accion_pendiente = 'eliminar'`.
- Al momento del `/sync`, la API de Python intercepta esta acción y ejecuta un `UPDATE suscripciones SET actividad = 0` en MySQL. Esto desactiva las alarmas de cobro pero mantiene intacto el historial de reportes.

6. Organización del Equipo y Gestión de Tareas

El desarrollo se llevó a cabo aplicando metodologías ágiles y control de versiones mediante Git/GitHub, logrando una carga de trabajo equitativa basada en las fortalezas de cada integrante:

Integrante	Rol Principal	Aportes Clave y Tareas Desarrolladas
Lorenzo Poletto	Arquitectura Backend, Nube y Controladores	Configuración del VPS, desarrollo de la API REST en FastAPI. Desarrollo del DataManager para orquestar la comunicación del sistema, gestionando los requests asíncronos en C++ y la sincronización con el servidor.
Santiago Agostini	Frontend UI/UX y Visualización	Diseño total de la experiencia de usuario. Desarrollo del StyleManager (QSS), diseño de pantallas responsivas (Dashboard, Reportes), componentes dinámicos (Cards) y validaciones de entrada.
Avril Ogas	Modelado de Datos y Persistencia	Creación de la estructura relacional (MySQL/SQLite). Desarrollo de la clase adminDB, implementación de la lógica de alertas tempranas de vencimiento y desarrollo de la arquitectura de borrado lógico (Soft Delete).

7. Conclusiones

El de **Alcancia** ha superado ampliamente los alcances de un sistema ABM (Alta, Baja y Modificación) tradicional, consolidándose como una plataforma financiera proactiva, resiliente y de nivel profesional.

El éxito del proyecto se fundamenta en tres pilares clave:

1. **Innovación Tecnológica:** La integración de la API de Inteligencia Artificial (OpenAI) para la extracción semántica de datos, junto con el soporte nativo para renderizado de PDFs, automatiza por completo la carga de comprobantes y elimina las fricciones del usuario final.
2. **Arquitectura Híbrida (Offline-First):** La implementación de una topología de red bidireccional garantiza una fluidez absoluta. El uso de una base de datos local (SQLite) como caché de alta velocidad, sincronizada asíncronamente con un motor transaccional en la nube (MySQL/FastAPI), dota al sistema de tolerancia a fallos y disponibilidad continua sin depender de la conexión a internet.
3. **Rigor Académico en POO:** El diseño del software demuestra un dominio sólido de los pilares de la Programación Orientada a Objetos. La correcta aplicación de abstracción, herencia, polimorfismo, encapsulamiento estricto y patrones de diseño estructurales (como el Singleton en el DataManager) permitió construir un código C++ modular, escalable y libre de redundancias.

En síntesis, Alcancia evidencia el crecimiento técnico del equipo al transicionar de los conceptos teóricos a la arquitectura y despliegue de un producto de software real.

Mediante una interfaz gráfica pulida (UI/UX) y la aplicación de buenas prácticas de bases de datos (como el borrado lógico o *Soft Delete* para preservar el historial), el resultado es un sistema robusto que excede con creces los objetivos planteados al inicio de la asignatura.